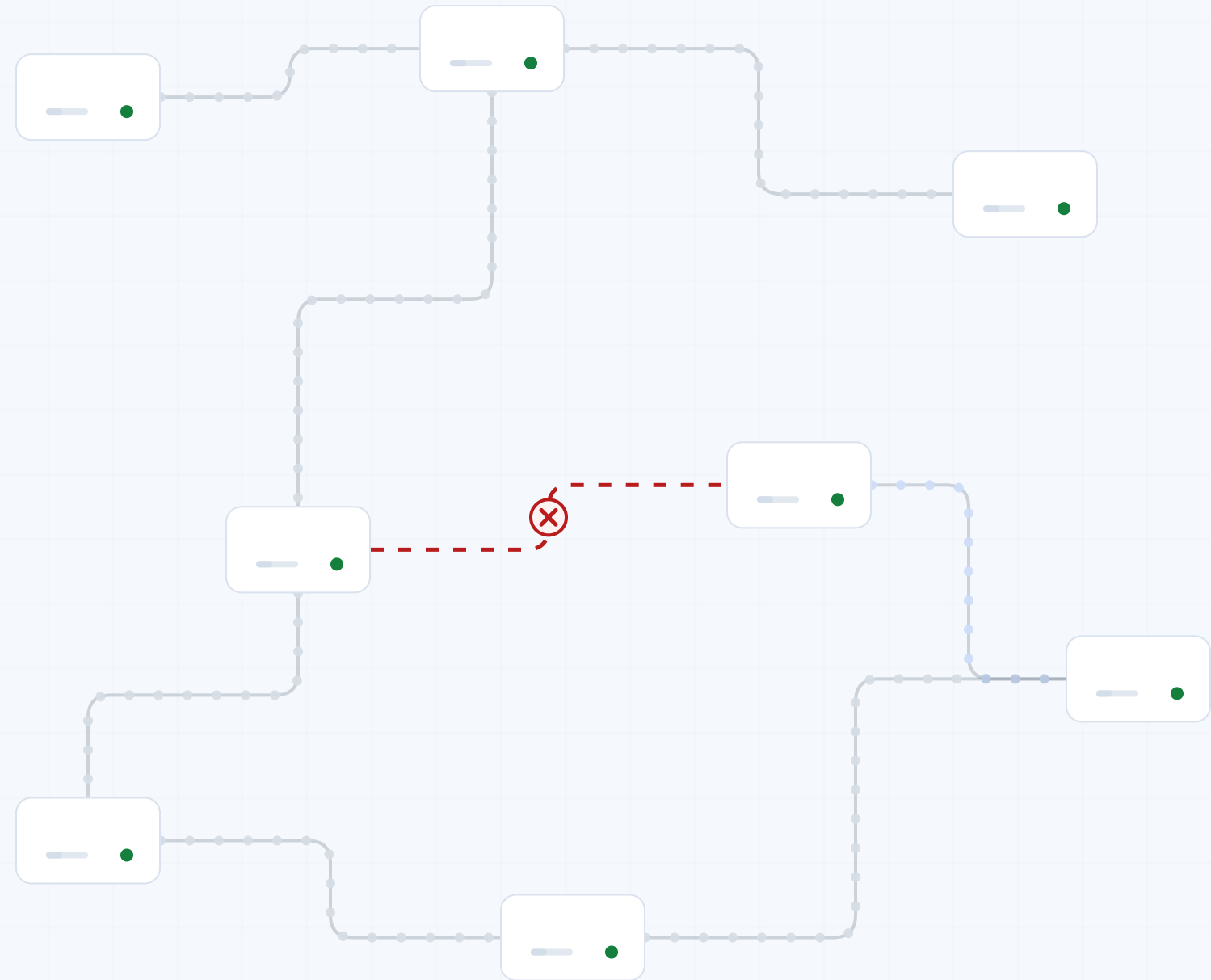


De compose

a orquestación.

LA PREGUNTA

¿Y si una sola máquina no basta? El cierre de la sesión pasada.



240 minutos,

una decisión.

CUATRO HORAS

La mañana es Compose, Kubernetes y una introducción a consenso, con una decisión antes del descanso; la tarde es Mexi Banco **en su máquina** y en un clúster.

AVISOS

- Equipos formulario esta semana, se validan en el Taller (S07)
- Examen sale en la S07, individual, **a mano y en casa**
- Hoy sí hay lab Mexi Banco con Compose, tres hitos



Lo que quedó de la S4.

CUATRO FRASES

Antes de la idea nueva, **reconexión**
en pares: un minuto solos, dos con
alguien más.

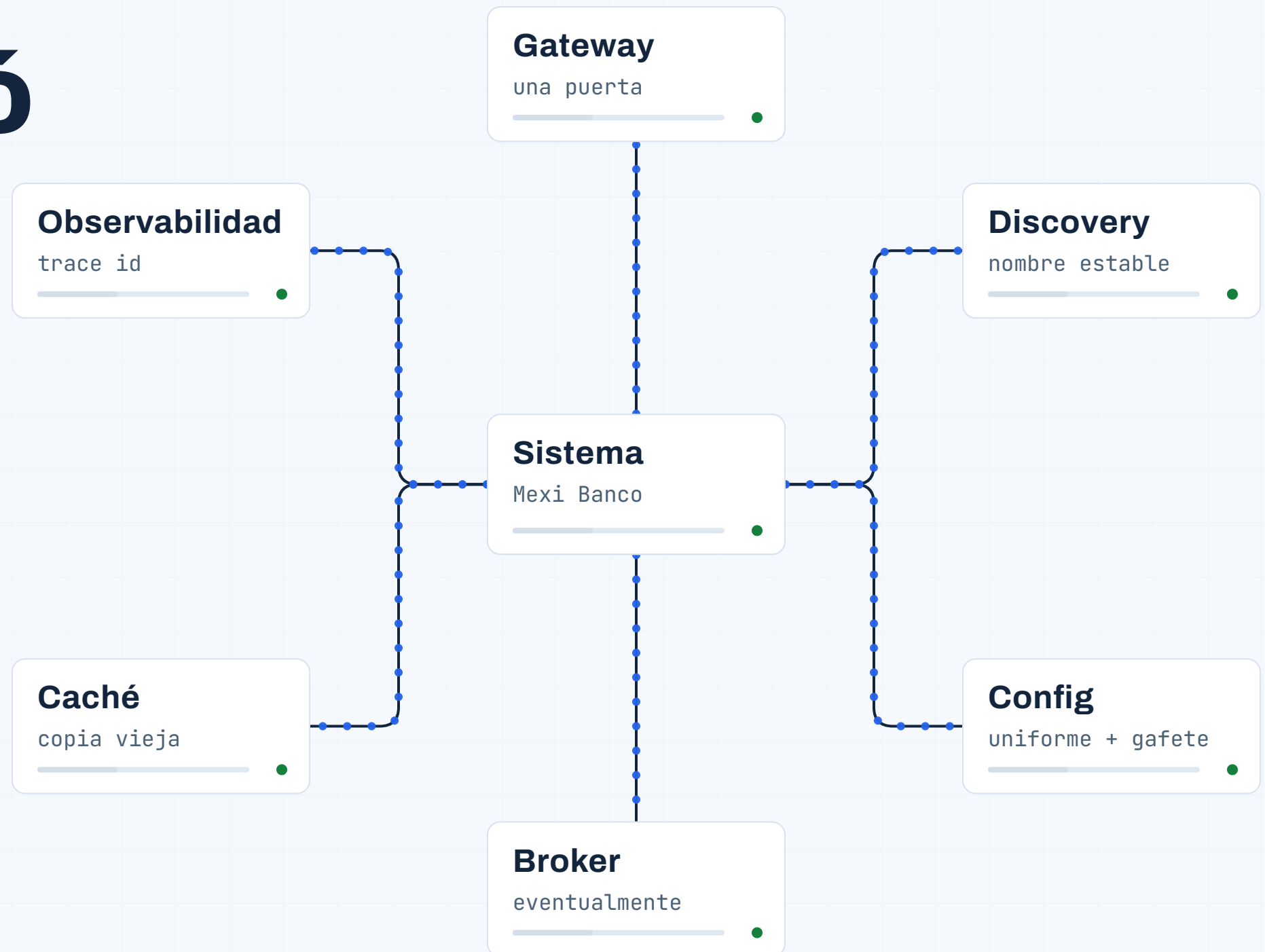
LO QUE YA SABEN

REST o gRPC el borde habla REST, el interior
gRPC

Factor III la config vive fuera de la imagen

Seis piezas **discovery** ya la usan sin decirlo

Cada servicio dueño de su almacén



¿Cuándo está

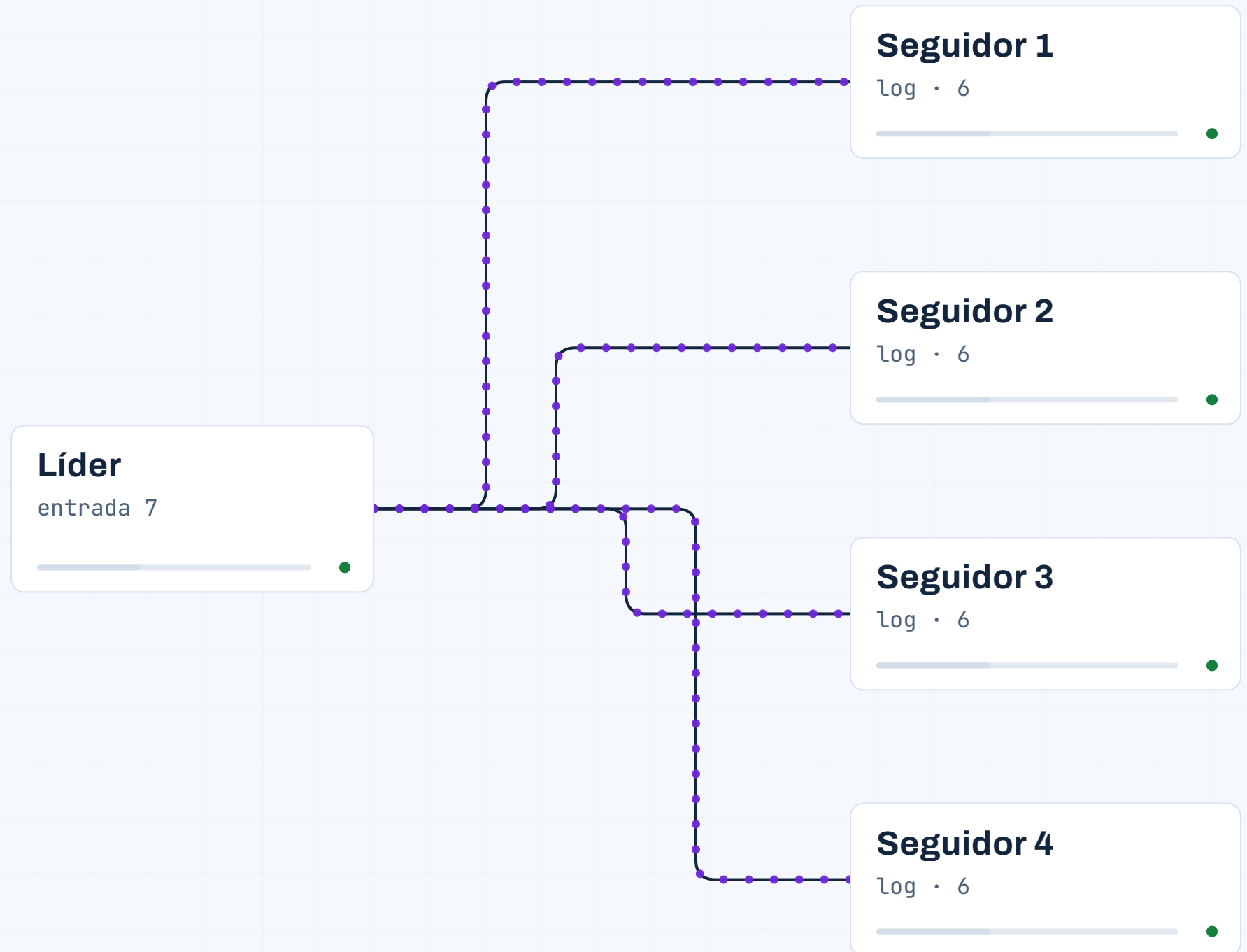
comprometida?

PREGUNTA DE LA SEMANA

Raft, 5 servidores. El líder manda una entrada al log. ¿Cuándo la considera **comprometida**?

ELIGE UNA

- A cuando la escribe en su propio log
- B cuando la mayoría, 3 de 5, ya la replicó
- C cuando los 5 la replicaron
- D cuando el cliente recibe la respuesta



El nombre es el hostname.

DNS EMBEBIDO

El nombre del servicio en `docker-compose.yml` es su **hostname** dentro de la red del proyecto.

LO QUE ESO DA

Una red bridge por proyecto, la crea Compose sola

`app` resuelve `db` **sin ninguna IP** escrita en ningún lado

Mismo principio que Eureka pero en una sola máquina

DNS ¿DÓNDE VIVE "DB"?

app

mexi-banco · :8080

jdbc://db:5432

db

postgres:17-alpine

Existir no

es responder.

DEPENDS_ON

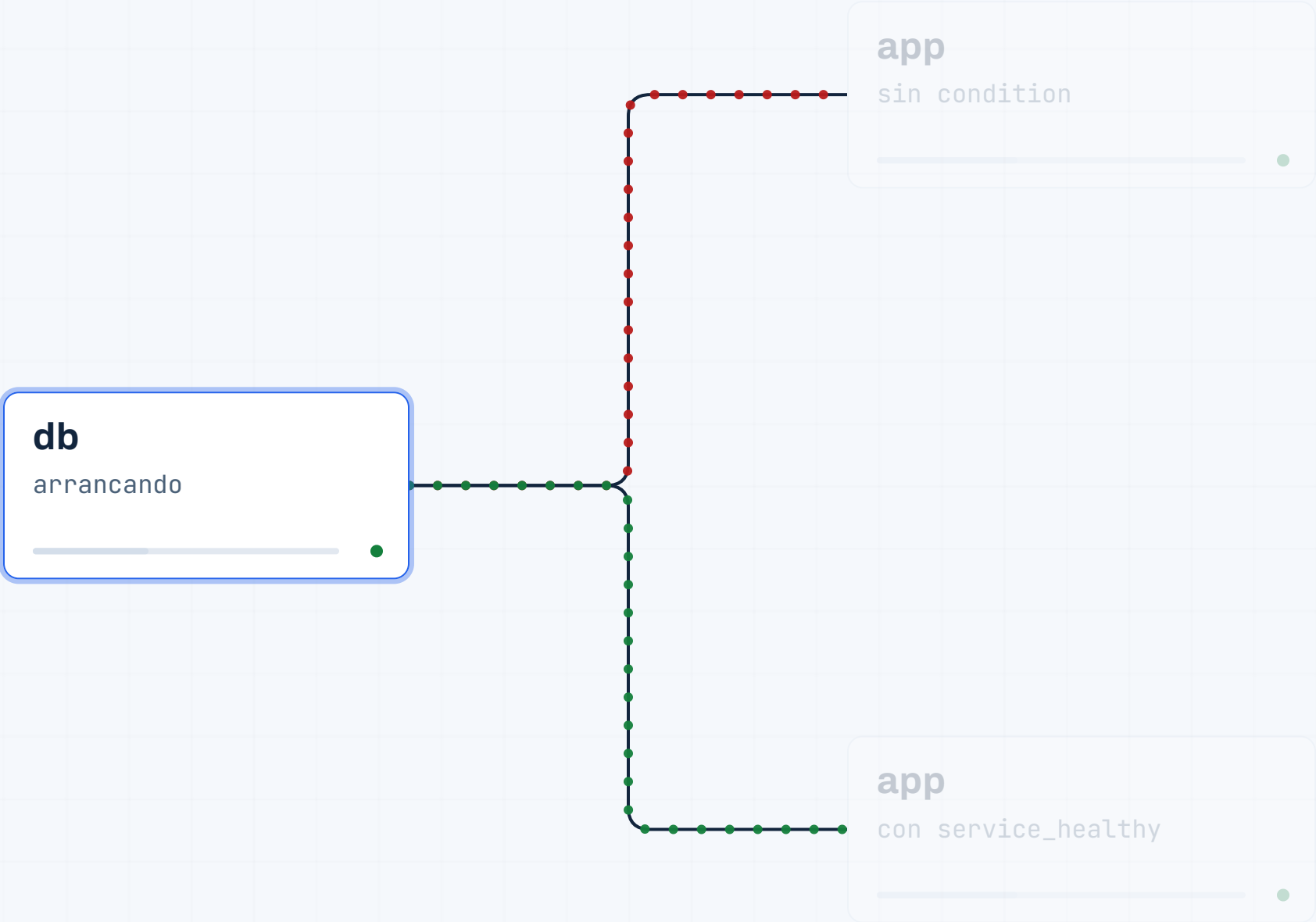
`depends\_on` ordena el **arranque**;
`condition: service\_healthy` espera a
que el servicio **responda**.

LA DIFERENCIA

Sin condition espera solo a que el contenedor exista

`service\_healthy` espera al healthcheck, **no al proceso**

El error clásico arrancar antes de que la base acepte conexiones



El gafete,

otra vez.

VARIABLES POR AMBIENTE

La misma imagen corre en tu laptop, en Compose y en el clúster; lo que cambia viaja por **variables de entorno**, nunca dentro de la imagen.

EN MEXI-BANCO

``DB_HOST: db`` en compose; en Kubernetes también vale ``db``

``${DB_HOST:localhost}`` en ``application.yml``: el default es para el IDE

Factor III ¿podrías hacer público el repo **sin exponer una credencial**?

mexi-banco

una sola imagen

Laptop

DB_HOST=localhost

Compose

DB_HOST=db

minikube

DB_HOST=db

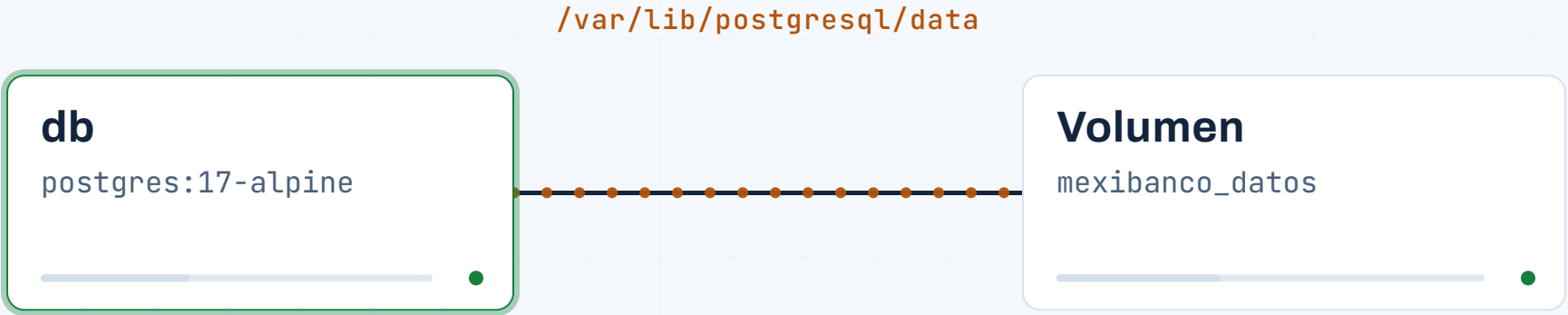
Lo que sobrevive.

VOLUMEN NOMBRADO

Un directorio que vive **fuera** del contenedor: tirar `db` no borra los datos; tirar el volumen, sí.

TRES COMANDOS

`docker compose down` conserva
`mexibanco\_datos`
`down -v` lo borra: cuentas y movimientos se van
Bind mount carpeta del host: útil en desarrollo,
frágil en producción



El compose de Mexi Banco,

línea por línea.

DOCKER-COMPOSE.YML

Dos servicios, una red y un volumen: todo lo de las tres láminas anteriores **en un solo archivo**, el que van a levantar en la tarde.

LAS CUATRO LÍNEAS QUE IMPORTAN

depends_on: db: condition: service_healthy

healthcheck: pg_isready -U mexibanco

environment: DB_HOST: db

volumes: mexibanco_datos:/var/lib/postgresql/data

RED MEXIBANCO · LA CREA COMPOSE

DB_HOST=db · service_healthy



¿Qué garantiza service_healthy?

COMPROBACIÓN 1

En ``docker-compose.yml``, ``app`` tiene ``depends_on: db`` con ``condition: service_healthy``. ¿Qué **garantiza** exactamente?

VOTA EN EL CHAT

- A** Postgres ya acepta conexiones al arrancar ``app``
- B** el contenedor de ``db`` existe y corre, nada más
- C** ``app`` y ``db`` comparten red, sin importar el orden
- D** si ``db`` se cae después, ``app`` se reinicia sola



Un solo edificio.

UNA SOLA MÁQUINA

Si el contenedor muere o la máquina se apaga, **no hay adónde migrar**.

LO QUE COMPOSE NO HACE

Sin `restart` no revive solo

`--scale app=3` tres puertos, nadie reparte

Siempre **la misma máquina**



UN SOLO HOST · DOCKER COMPOSE UP --SCALE APP=3

Desarrollo

contra producción.

COMPOSE ES DESARROLLO

Un `docker compose up` y todo arranca. Lo que producción pide, Compose **no lo trae**.

LO QUE PIDE PRODUCCIÓN

Autocuración detectar la falla y reponer, sin llamar a nadie

Escalado réplicas declaradas, no puertos a mano

Balanceo alguien que reparta entre las copias

Máquinas **más de una**, y que una pueda faltar

Compose

una máquina

NADIE REPONE AL CAÍDO

ESCALA A MANO, POR PUERTO

SIN BALANCEO

UN SOLO HOST

Kubernetes

un clúster

REPONE EL POD

REPLICAS: 3

SERVICE REPARTE

N NODOS

Di qué quieres, no cómo.

MODELO DECLARATIVO

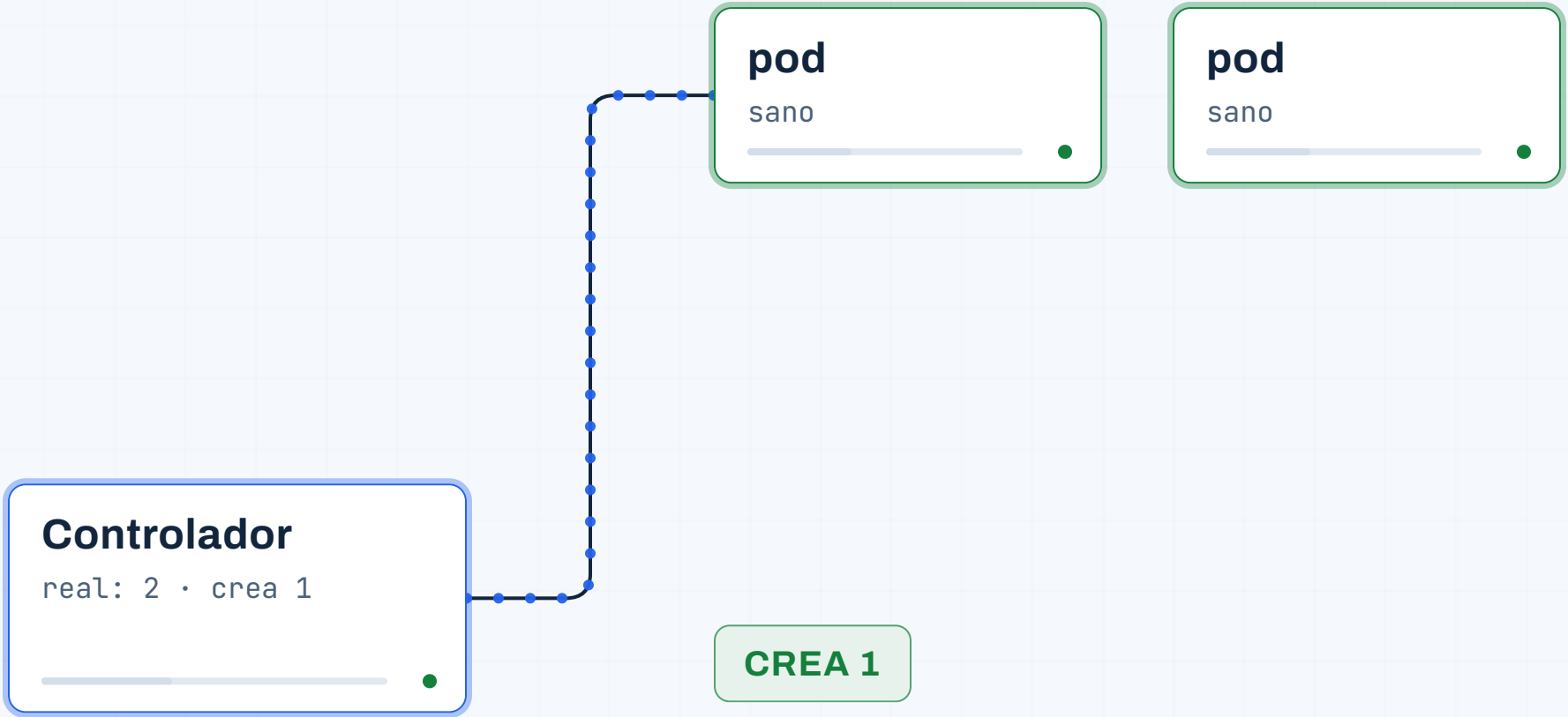
Tú declaras el estado deseado; un controlador compara sin parar con el real y **corrige la diferencia**.

"QUIERO 3 RÉPLICAS"

Hay 2 crea 1

Hay 4 borra 1

Una muere **crea otra**, sin que nadie llame



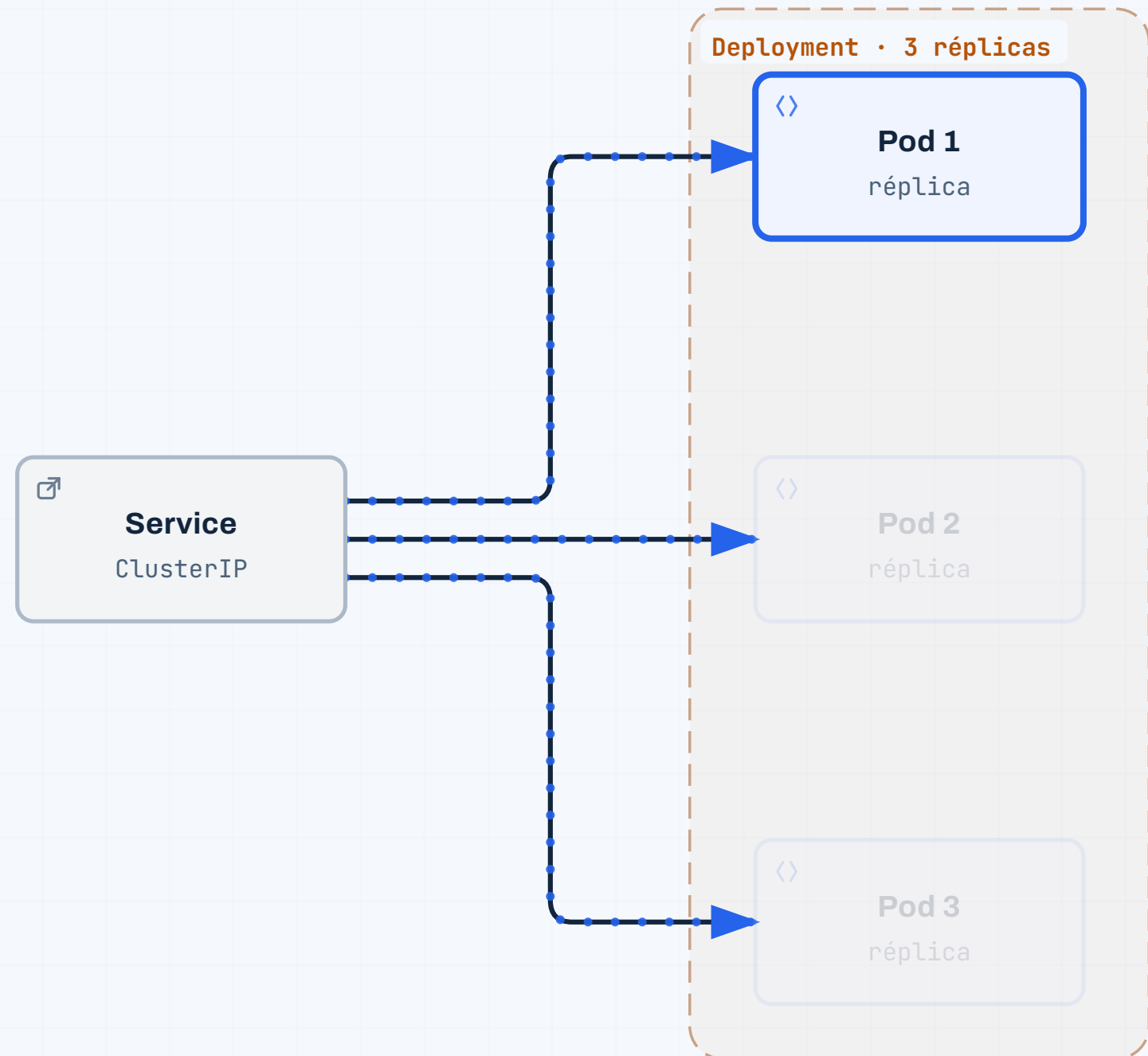
Tres piezas, no más.

POD · DEPLOYMENT · SERVICE

La copia desechable, quien **cuenta** las copias, y la puerta estable hacia ellas.

HOY SOLO ESTO

Pod unidad mínima, desechable, con IP propia
Deployment declara cuántas copias debe haber
Service nombre DNS y balanceo estables



Dos piezas de fábrica.

DNS Y VARIABLES

El nombre de un Service ya es DNS y las variables del Deployment ya son config: **dos de las seis piezas** vienen con el clúster.

ANTES Y DESPUÉS

Eureka ``lb://CUENTAS`` pasa a ``http://cuentas:8080``

Config server pasa a ``env:`` en el Deployment

La S06 las construimos nosotros, **sin clúster**, para entender qué hacen

Eureka

directorio · late 30 s

Service

DNS del clúster

Config server

git + refresh

env:

en el Deployment

La imagen viaja.

MULTISTAGE

Etapa 1 compila con JDK y Maven; etapa 2 copia **solo el `.jar`** sobre un JRE. Lo que se manda al clúster no trae ni compilador ni código fuente.

PARA QUE MINIKUBE LA VEA

`minikube image load` la imagen local no entra sola al clúster

`imagePullPolicy: Never` no busca en ningún registry

Cache de `:latest` si reconstruyes, `kubectl rollout restart`



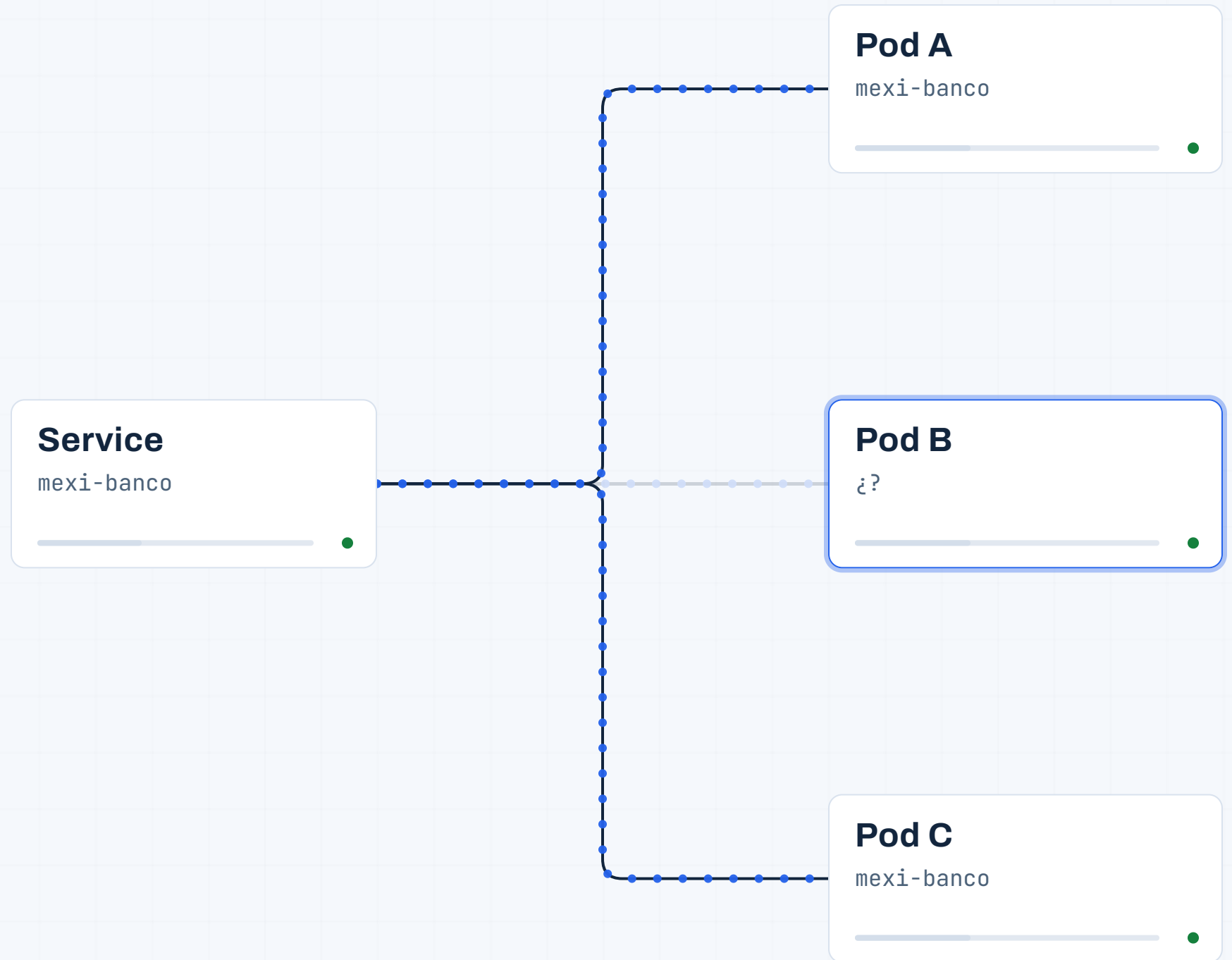
¿Qué predices?

COMPROBACIÓN 2

Esta tarde: un Deployment de Mexi Banco con 3 réplicas. Matas un Pod con `kubectl delete pod`. ¿Qué **predices** que pasa?

VOTA; LO REVISAMOS EN LA DEMO

- A** el mismo Pod se reinicia, mismo nombre
- B** el Deployment crea un Pod nuevo, otro nombre, AGE en cero
- C** Kubernetes lo restaura desde una copia de seguridad
- D** el clúster completo se reinicia



Un latido, y si falta, elección.

HEARTBEAT

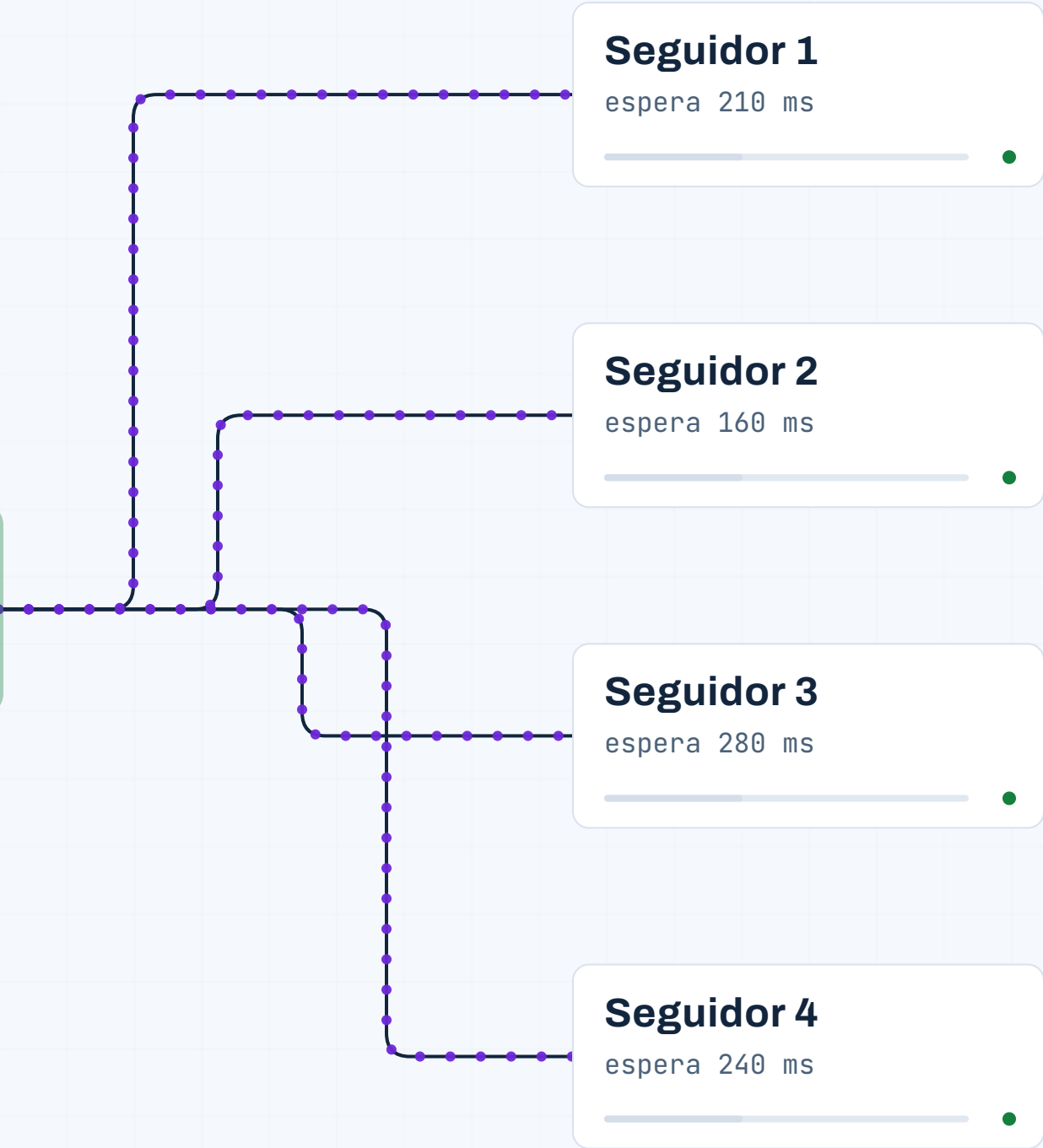
El líder manda un **latido** periódico.
Un seguidor que no lo oye en su tiempo de espera deja de esperar, se postula y pide votos.

LOS NÚMEROS DE RAFT

- Latido cada 50 a 150 ms; Eureka late cada 30 s
- Timeout aleatorio 150 a 300 ms: **no todos se postulan a la vez**
- Líder nuevo quien junte la mayoría de votos

Líder

latido · 100 ms



La mayoría decide.

QUÓRUM

Más de la mitad de los nodos. **Dos mayorías siempre comparten un nodo**: no puede haber dos líderes ni dos verdades.

LO QUE SE DESPRENDE

3 de 5, 2 de 3 con eso se elige y se compromete

Impares 4 nodos toleran 1 caído, **igual que 3**: el cuarto no compra nada

etcd por eso corre con 3 o 5 réplicas

Nodo 1

copia del estado

Nodo 2

copia del estado

Nodo 3

copia del estado

Nodo 4

copia del estado

Nodo 5

copia del estado

MAYORÍA A · 1, 2, 3

Quién sostiene el acuerdo.

ETCD

etcd guarda el estado del clúster (cuántos pods debe haber y cuántos hay); sus réplicas acuerdan con Raft.

EL GANCHO

Kubernetes por dentro lo que acaban de ver, corriendo en el control plane

3 o 5 réplicas de etcd en producción: mayoría

La S06 Raft a fondo: el log replicado y qué pasa al cambiar de líder



El mapa completo.

COMPOSE CONTRA KUBERNETES

Mismo modelo declarativo, **distinta escala**: un archivo para una máquina; un clúster que se repara solo y acuerda por mayoría.

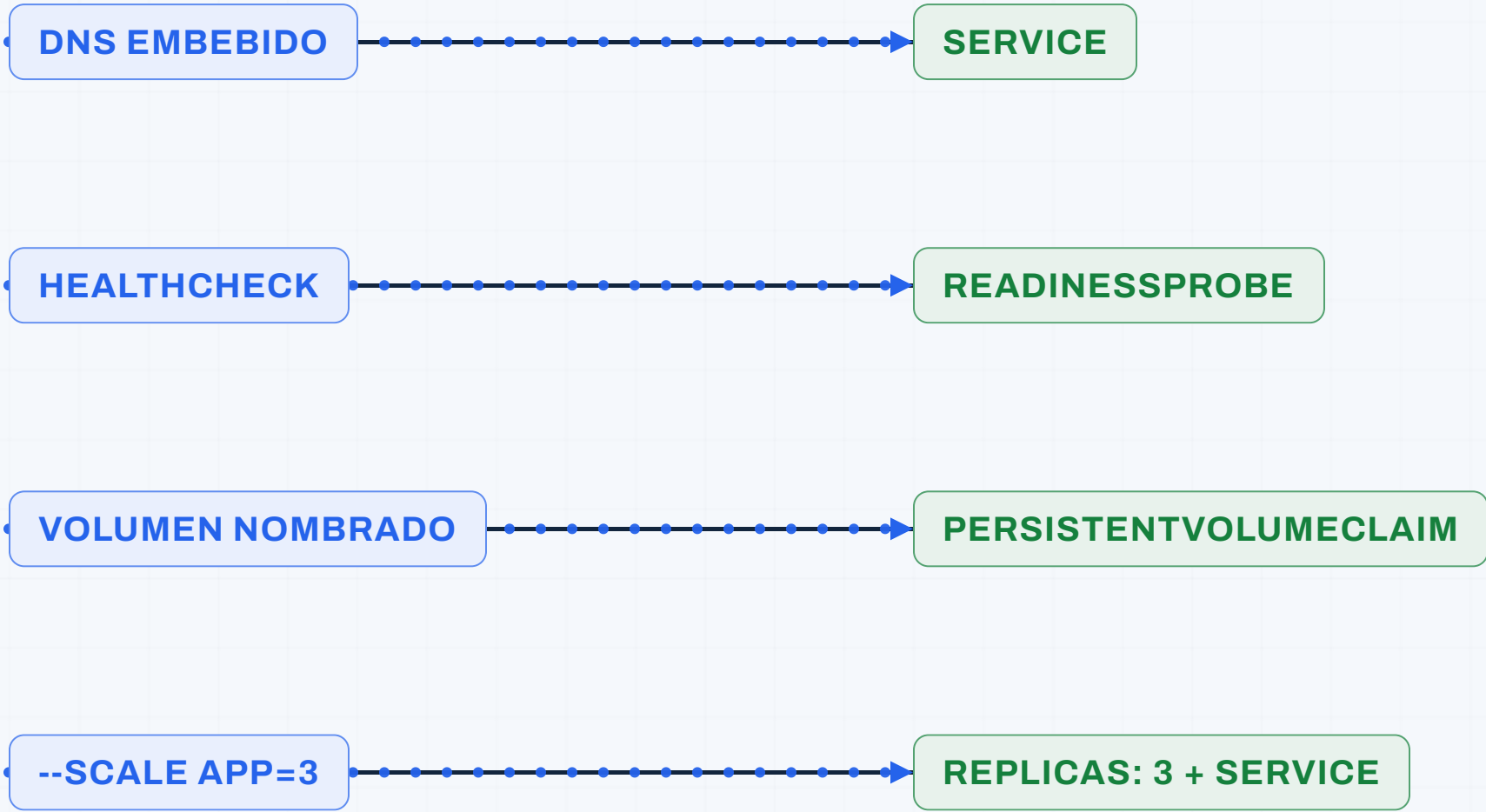
CADA IDEA TIENE SU PAR

Nombre estable DNS embebido · Service

Salud healthcheck · readinessProbe

Datos volumen nombrado · PersistentVolumeClaim

Copias `--scale` sin balanceo · `replicas` con Service



La decisión

de hoy.

LA DECISIÓN

Réplicas o máquina más grande:
Mexi Banco recibe el triple de tráfico de golpe en la autorización de tarjeta.

EL RITUAL, EN TRES LÍNEAS

- Elige una de las dos
- Di por qué la razón real, con el requisito enfrente
- Di qué sacrificas lo que pierdes al elegir

Mexi Banco

autorización · <100 ms

réplica

igual tamaño

réplica

igual tamaño

réplica

igual tamaño

1 instancia

más CPU / RAM

22 / DESCANSO

Regresamos

a las 09:20.

DESCANSO

30 minutos. Regresamos a las **09:20** con Mexi Banco corriendo en su máquina.

MIENTRAS TANTO

`git clone` el repo mexi-banco, liga en el chat
`docker compose up --build` la primera vez tarda unos minutos

Si algo truena lo vemos a las 09:20, **no lo peleen solos**

clone

mexi-banco · v05

up --build

db + app

healthy

compose ps

Un banco, cinco piezas.

UN MONOLITO

Cuenta, movimiento, transferencia, SPEI y notificación, en un solo **deployable**: hoy es código, no solo un ejemplo hablado.

LOS CINCO

- Cuenta CLABE y saldo
- Movimiento asiento inmutable: el ledger solo crece
- Transferencia interna, una sola transacción
- SPEI externo, lento e irreversible

MEXI-BANCO V05 · UN SOLO DEPLOYABLE · UNA SOLA BASE



Qué se puede pedir.

CUATRO ENDPOINTS

Abrir, consultar, transferir, enviar por SPEI. Y `/actuator/health`, que es lo que el healthcheck y las probes preguntan.

REST, COMO EN LA T01

- POST /cuentas
- GET /cuentas/{clabe}
- POST /transferencias
- POST /spei + Idempotency-Key



Un sistema

que crece.

UNA PIEZA POR SÁBADO

Hoy es un monolito. Cada sábado le agregamos **la pieza que ese día se explica**, igual que ustedes en su proyecto.

LO QUE VIENE

- S06 se parte en tres; aparecen la puerta y el directorio
- S08 · S09 resiliencia y trazas · caché del saldo
- S10 · S12 eventos y notificaciones reales · la saga del SPEI



Tres hitos,

con tus manos.

CON TUS MANOS

Arriba y respondiendo, las propiedades de Compose, la idempotencia otra vez. **Cada hito termina con algo que se ve.**

ENTREGA, ANTES DEL DOMINGO

``docker compose ps`` ambos servicios ``healthy``
Segundo intento de SPEI misma clave, sin doble cobro
README las tres preguntas de cierre

Hito 1 · Arriba y respondiendo

`compose up --build` · abrir y consultar una cuenta · 20 min

Hito 2 · Propiedades de Compose

`network inspect` · transferencia · matas app y no revive · 20 min

Hito 3 · Idempotencia otra vez

misma SPEI, misma clave, un solo cobro · 15 min

Se repara solo.

AUTOCURACIÓN

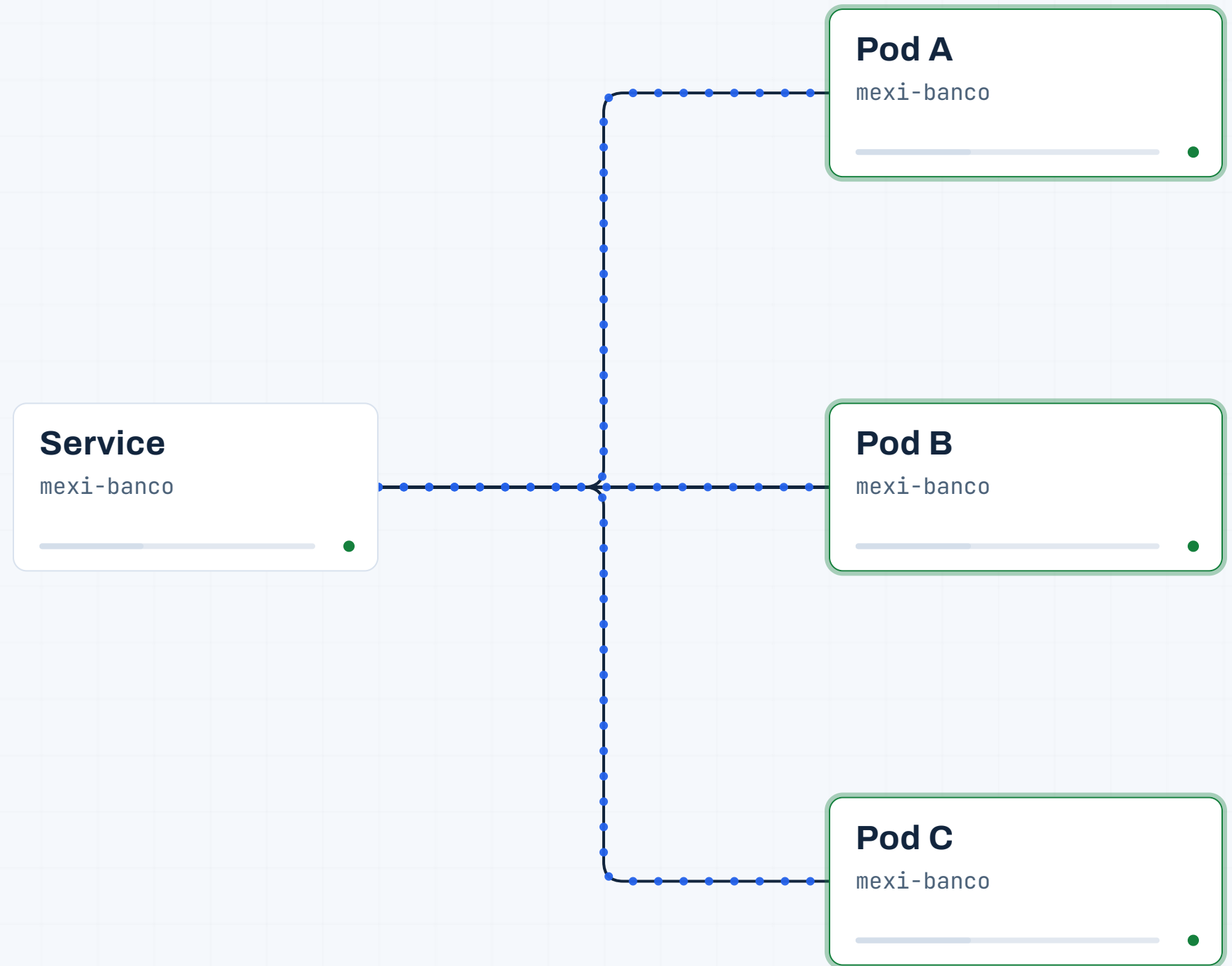
El Deployment no revive el Pod: crea uno **nuevo** porque nota que falta una copia. El mismo Mexi Banco que acababan de correr.

LO QUE VAN A VER

Mismo Deployment 3 réplicas declaradas en `k8s/app.yaml`

Pod nuevo otro nombre, AGE en segundos

Nadie llamó a nadie es reconciliación continua



Una puerta y un directorio.

PARA LA S6

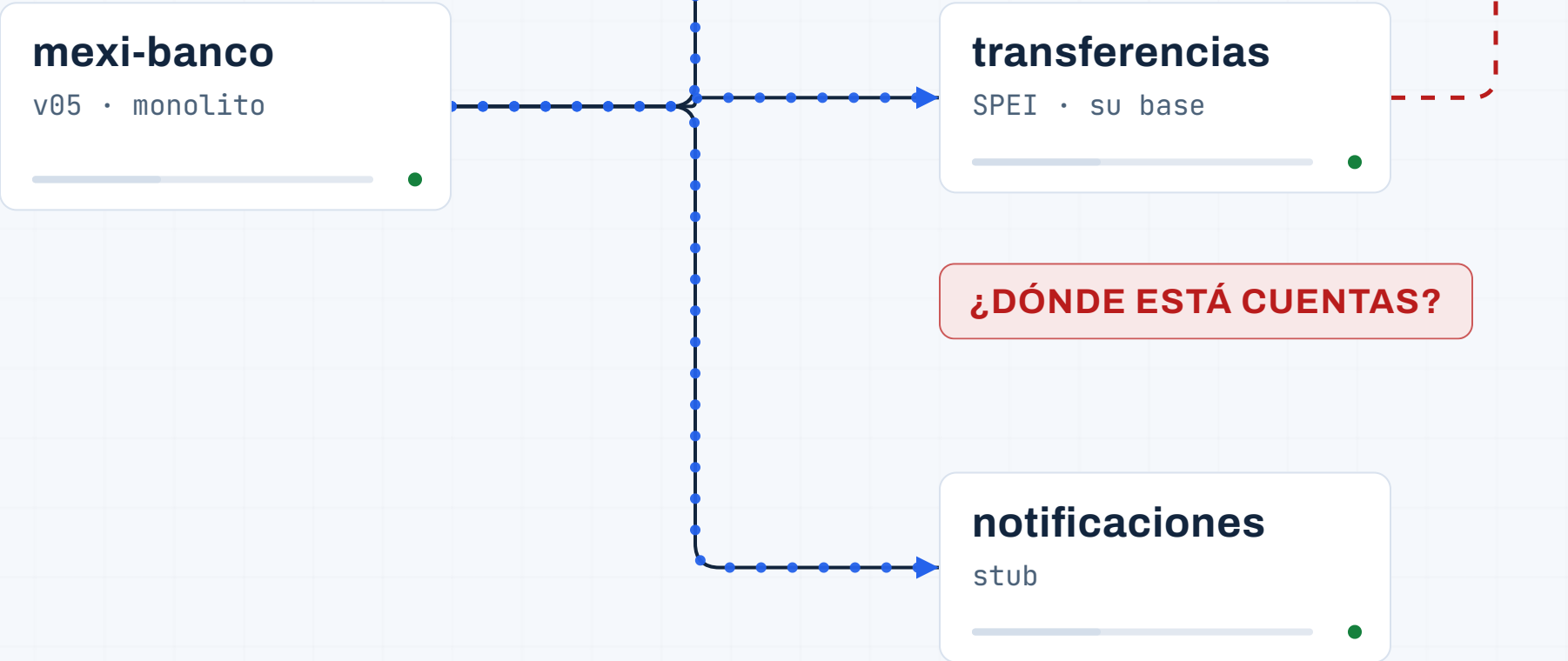
Raft a fondo, consistencia y CAP. En el lab, Mexi Banco se parte en tres y la transferencia **deja de funcionar** sin gateway ni discovery.

ANTES DEL SÁBADO 26

Lectura 5 Brewer, CAP Twelve Years Later (2012); Discussion el jueves

Traer Mexi Banco corriendo en Compose, como hoy

Equipos formulario esta semana; se validan en el Taller



Ocho preguntas

que ya puedes responder.

OCHO PREGUNTAS

Si puedes contestarlas sin ver notas, **la sesión quedó**. Si una no, esa es tu tarea de la semana.

COMPOSE

- 1 ¿Por qué ``app`` encuentra a ``db`` sin ninguna IP?
- 2 ¿Qué garantiza ``depends_on`` sin condition, y con ``service_healthy``?
- 3 ¿Qué le pasa a un contenedor caído en Compose sin ``restart``?
- 4 ¿Por qué ``--scale app=3`` no basta para escalar?

KUBERNETES

- 5 ¿Qué es un Pod, un Deployment y un Service, en una frase cada uno?
- 6 ¿Qué pasa exactamente cuando matas un Pod de un Deployment?
- 7 ¿Qué dos de las seis piezas trae el clúster de fábrica, y qué las reemplaza?
- 8 ¿Qué hace un seguidor cuando deja de oír el latido, y por qué dos mayorías de 5 siempre se cruzan?

30 / CIERRE

Fuentes

de la sesión.

FUENTES

Todo lo de hoy sale de fuentes **primarias**: documentación oficial y los papers de la semana.

DOCUMENTACIÓN

Docker Compose docs.docker.com: depends_on, healthcheck, volumes

Kubernetes kubernetes.io/docs: Pods, Deployments, Services

12factor.net Wiggins, 2011: factor III

PAPERS Y LIBROS

Ongaro y Ousterhout, 2014 In Search of an Understandable Consensus Algorithm (Raft), USENIX ATC

Burns, Grant, Oppenheimer, Brewer y Wilkes, 2016 Borg, Omega, and Kubernetes, ACM Queue

Verma et al., 2015 Large-scale cluster management at Google with Borg, EuroSys

Burns, Beda, Hightower y Evenson, 2022 Kubernetes: Up and Running, 3.^a ed., O'Reilly

Ya lo comprobaste.

EL PRINCIPIO

Compose asume que la máquina no falla; **Kubernetes** asume que sí, y automatiza la reacción. Y ya lo comprobaste con tus manos.

ANTES DE CERRAR

Ticket de salida ¿acertaste tu predicción? ¿qué fue lo más confuso?

Lectura Raft §1-5, jueves 23:59, 2 preguntas + 1 hallazgo

Próxima S06 · Raft a fondo, consistencia y CAP; Mexi Banco en tres

